

SSY098 Project Descriptions 2026

- This document contains four project proposals. Each project has 40 available spots, and you may sign up for your preferred project via Canvas. Allocation is on a first-come, first-served basis, and you are free to choose any of the available options. Sign-up opens on **Monday, May 18 at 15:00**. Each project has its own supervisor (TA) who will be responsible for its supervision:
 - Project I: Analysis of a Classifier - Vilgot
 - Project II: Memory - Victor
 - Project III: Structure-from-Motion in 2D - Sofie
 - Project IV: Diffusion Models - Josef
- Each project consists of three parts: (i) A mandatory part, (ii) a theoretical part, and (iii) an advanced part.
 - In order to pass the project with a grade of 3, you need to complete the mandatory part.
 - Completing the mandatory part (i), and either the advanced part (ii) **or** the theoretical part (iii), will allow you to pass the project with a grade of 4.
 - Completing the mandatory part (i), and both the advanced part (ii) **and** the theoretical part (iii), will allow you to pass the project with a grade of 5.
 - The theoretical part (iii) consists of three questions, each of which is worth 1 point. You need 2 points to pass the theoretical part. Your answers will be graded on a continuous scale, i.e., we try to reward partially correct answers.
- Submission:
 - You should submit a Python notebook file and an HTML printout. Submit all files through Canvas.
 - In addition to your Python code, you will need to write a short report describing your project and providing experimental results. If you submit answers to the theoretical questions, include your answers in the report as well. The report should also include an AI usage statement (mandatory). Please use the IEEE conference template found [here](#) for your report. Submit your report as a PDF file through Canvas.
- **Note:** Projects must be done individually and may not be carried out in groups. No collaboration is allowed, and you are only permitted to ask for help from the teaching assistants or the lecturer. Copying solutions, even partially, from any source is strictly prohibited. If you use any non-human source beyond the course material and literature, such as a research article or a book, you must include a proper reference. You may use LLMs, but your report must contain a clear, complete, and truthful description of how they were used, and you must understand and be able to explain everything you submit.

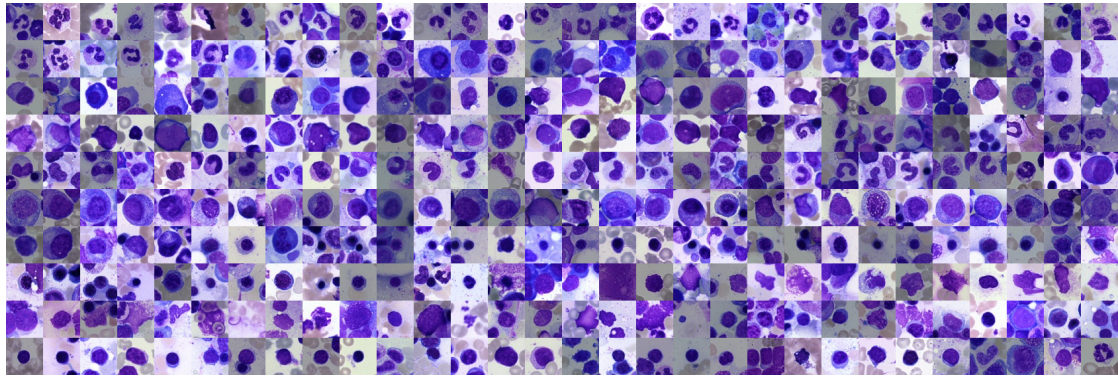


Figure 1: Some samples from the Bone Marrow Cell Classification dataset.

1 Project I: Analysis of a Classifier

The goal of this project is to develop a model to classify images of bone marrow cells. This will be done with an at least partially convolutional neural network, which is then further improved.

1.1 Mandatory Part

Design a convolutional neural network, optionally with some fully connected layers, and train it on a dataset derived from the **Bone Marrow Cell Classification dataset**. The dataset was created by selecting a subset of the original images and resizing them, giving you a balanced set of classes. Training and test sets were separated randomly. Download [here](#). This file contains a train and a test directory, both then containing eight directories representing classes¹. Elements of the dataset are $56 \times 56 \times 3$ colour images in the JPEG format with values from 0 to 255². There are 32000 train and 24000 test images.

Even a fairly shallow model should be able to reach 70% accuracy on the test data with 15 – 20 epochs of training. If you have a CUDA-enabled GPU (or a lot of time), you can also try for a deeper model. Once you have this, evaluate with **confusion matrices**, training plots, and any further evaluation methods of your choice where your original model is lacking. Based on this choose **at least three** methods to improve your model in these aspects:

- **Pre-processing:** Modifying the data in some non-learned way can often improve training significantly. For example, you could use **PCA** to extract the dimensions carrying the most information.³
- **Early stopping:** Use a validation set split from the training data to decide when to stop training. The method **cvpartition** may come in handy for doing this in a stratified way.
- **Batch Normalisation:** Including batch normalisation layers before some or all of your activation functions often makes training markedly easier.

¹The **ImageFolder** class might be helpful for loading this data.

²A different range might work better for your model. I suggest to already consider this for the base network, perhaps even apply **standardisation** (based on training data only!).

³Neural networks are theoretically able to implement the same transformations as PCA themselves, but applying them manually can help in some cases.

- **Optimisers and Activation Functions:** The choice of optimisers, activation functions, and their hyper-parameters is crucial for training. While Adam and ReLU are often solid options, it can pay to try others.
- **Data Augmentation:** Use rotations, flipping, cropping, or any other methods to create additional training images from the data you already have.
- **Other:** You are encouraged to research other methods on your own.

Explain in detail why you chose these specific methods. With this you should reach a test accuracy of 81% or more⁴. Finally, do an **ablation study** with your improvements. I.e., compare your final model to additional models, each including all but one of your improvements, showing you what impact the absence of a given improvement has. This should be done based on test accuracy, as well as confusion matrices for all models (as well as any other metrics you consider useful). Running an ablation study can take a bit of time since you will have to do multiple runs. Start early and queue multiple trainings so you don't need to babysit it every time one training ends. If you need compute resources that can be left running you can try using Google Colab or any other cloud coding service.

What to include in the report. Your report should consist of the following:

- A short abstract describing the problem behind the project and outlining your solution.
- A “Method” section describing your approach: What network architecture are you using? What improvements did you choose and what problems are they supposed to solve? This section should be about one page long.
- A “Experimental Evaluation” section describing the experiments you perform with your approach as well as the results of these experiments:
 - Describe the parameters you used for training, i.e., your optimiser, the learning rate, whether you used momentum, etc.
 - Show the improvements you used. E.g., for early stopping plot the training and validation accuracy during training, for pre-processing visualise how you changed the images, etc.
 - Include the results of your ablation study. Do you see any pattern in the confusion matrices? Did your improvements achieve what you wanted them to? Compare what happened to the expected improvements you described in the “Method” section.
 - Choose a subset of test images and visualise (e.g., with a border) which ones were classified correctly and which one weren't. Explain why you think they were classified this way.

1.2 Advanced Part

To classify objects within a larger image, it is important to detect if a given frame contains an object at all. There are many methods to do so, most of which involve training the model with this capability to begin with. However, for the advanced part here, you will instead extend the previous classification network without any new training.

⁴Note that normally you, of course, shouldn't attempt to improve a model based on test data results, as that may introduce a bias toward that dataset. Each phase would need each own new test data afterwards. However, it does commonly happen in practice, as there is often limited data available.

Specifically, your task is to separate the 2000 images provided [here](#) into in-distribution (ID) samples (from the earlier training and test set) and out-of-distribution (OOD) samples (in this case images of [blood cells](#)). There are no labels provided so you will need to plot the images to see what kind of OOD samples we are working with.

You should not modify the network or train a new one, but rather use the information you can get from the activations of your network. Try to find a way to create a scalar score for each sample, usually from applying some function to the activations of one of the later layers but not necessarily the last. You are likely successful if you manage to get two distinct hills in a [histogram](#) of all the scores. Can you also figure out which hill corresponds to ID and which to OOD data⁵?

What to include in the report. Extend your report to include the following:

- Extend the “Method” section by a subsection describing how you searched for useful features, and what you did with them to detect OOD samples.
- Extend the “Experimental Evaluation” with a histogram showing the distribution of scores, colouring and labeling it according to which parts you believe to be ID/OOD data. Since there are no labels and therefore no test accuracy to refer to, you must figure out own way to demonstrate the accuracy of your labeling.

1.3 Theoretical Part

Answer the following questions in your report. Use your own words and justify your answers.

- (a) Compute the theoretical number of learnable parameters in your model. Show explicitly how the parameter count is derived for each relevant layer or component. Compare your theoretical result with the parameter count reported by the implementation, framework, or documentation, and discuss possible reasons for any differences.
- (b) Estimate the computational complexity of your pipeline. Clearly state all assumptions, including implementation details. Discuss which parts of the pipeline dominate the computational cost. Suppose the computational complexity had to be reduced by approximately a factor of 10 while preserving accuracy as much as possible. Propose and motivate one or more modifications to your pipeline.
- (c) In this project, the primary ablation strategy is based on holding out one component at a time from a more complete pipeline. Discuss what types of conclusions can and cannot be drawn from: (i) removing one component at a time, and (ii) adding one component at a time starting from a minimal baseline. Relate the discussion explicitly to your own experiments and results. Discuss situations where these different ablation strategies may lead to different interpretations of the importance of a component. Identify one question about your model that is still not fully answered by your current ablation study. Propose a new ablation experiment that could help answer this question.

⁵Hint: You are allowed to use known data from the base part, but do not directly compare the images to find the inliers.

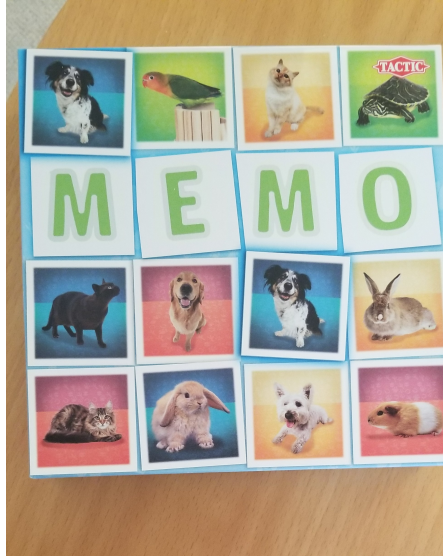


Figure 2: The card game “Memory”.

2 Project II: Memory

The goal of this project is to develop parts of a vision system for playing “Memory”, or “Concentration”. The system will act as a guide and keep track of all matching pairs. The formal rules for the card game are as follows.

Collect as many matching card pairs as possible, that is, two cards with the same picture. Spread the cards face down on the table. One player starts and the turn passes clockwise. On each turn the player turns two cards, so that everyone can see the pictures on them. If the cards match, the player takes them. He may continue until he turns two cards that don't match. When this happens, he turns the cards back face down and the turn passes to the next player. The player who has the most pairs at the end of the game wins.

You will take your own images for the project, so you need for instance a mobile phone camera. It is allowed to collaborate between yourselves to share cards, but keep in mind that you may want to test other things than your friends and that you may not copy each others' solutions or reports. It is allowed to make either a learnt solution (using e.g. neural networks) or a classical approach, bearing in mind the limitations of each method.

2.1 Mandatory Part

For this project you will implement the vision system of memory, *not the actual game mechanics*. The vision system should have a number of capabilities in order to assist playing the game, and should

1. detect the location of all cards in an image,
2. detect which cards are face up and which cards are face down,
3. determine if two cards depict the same picture or not.

The scene will be an image with 16 cards (8 pairs) face down, arranged in 4×4 grid, similar to Figure 3.

2.1.1 Data

Test set: To test your system, you should take a set of test images depicting the following situations:

1. All cards face down.
2. All cards face up.
3. One matching pair face up.
4. Two matching pairs face up.
5. One non-matching pair face up.

An appropriate amount would be at least 3 images of each.

Training set: You should also take a number of images on which to develop your solution. These may be as many or few as you wish, as long as you motivate it.

2.1.2 What to include in the report

Your report should consist of the following:

- A short abstract and introduction describing the problem behind the project and outlining your solution.
 - A “Method” section describing your approach in a free text, not bullet lists. Use a LaTeX “algorithm” environment of your choice if you want to outline your algorithm.
 - A “Experimental Evaluation” section evaluated on the test set. Note that the detected cards of interest should have bounding boxes as described in Figure 3, Figure 4 and Figure 5. You must also present appropriate quantitative metrics measuring how *i*) well cards are identified, *ii*) how well face-up and face-down cards are detected, and *iii*) how well pairs or non-pairs are identified.
- . You must present visual examples of your results in the report, and discuss your solution.

2.2 Advanced Part

In the advanced part, you should extend the functionality of the system in order to handle arbitrary rotations of the cards. Hence, unlike Figure 3, where the cards are organized in a 4×4 grid, the cards can be spread out arbitrarily on the table. We will assume that no cards are overlapping, and as before, when a card is turned, it is put at (approximately) the same location. You should also demonstrate that your solution is robust to (small) camera pose changes, so that the camera is not viewing the cards from above (that is, the image plane is not parallel to the plane with the cards). If you wish to do the advanced part, the test set must be updated to include these rotations and pose changes.

For the report, you should report the same things as in the mandatory part, but you should of course additionally include how you solved the problem of unknown orientation of the cards.

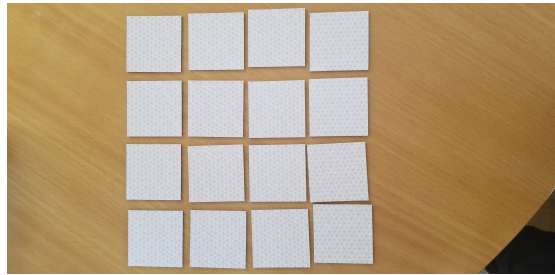


Figure 3: The card game “Memory”, starting board. Your system should locate and draw bounding boxes as output.



Figure 4: A matching pair! Your system should locate and draw **green** bounding boxes on the matching pair.

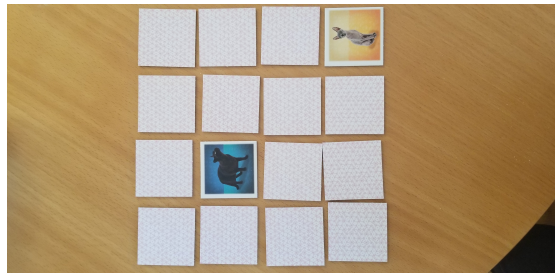


Figure 5: No matching pair! Your system should locate and draw **red** bounding boxes on the non-matching pair.

2.3 Theoretical Part

Answer the following questions in your report. Use your own words and justify your answers.

(a) Discuss how the choice of evaluation metric affects the interpretation of your results. Which aspects of the problem are well captured by your chosen metric(s), and which aspects are not? Discuss situations where two methods could obtain similar performance metrics while failing in qualitatively different ways. Support your discussion using concrete examples from your own experiments that support your claims, including both image examples and corresponding quantitative metrics.

(b) Suppose you doubled the number of different card motifs while keeping the total number of training images unchanged. Analyze which parts of your pipeline would likely degrade first and explain why. Support your discussion using explicit examples from your own implementation and collected data.

(c) Analyze 2-3 implicit assumptions made by your method and dataset. For each discussed assumption: (i) identify explicit examples from your own dataset where the assumption holds, (ii) identify situations where the assumption may break, and (iii) discuss how this would likely affect performance. Discuss which modifications would most likely improve robustness without substantially increasing computational complexity.

3 Project III: Structure-from-Motion in 2D

In this project, you will implement a Structure-from-Motion pipeline in 2D, which is equivalent to stitching together multiple 2D images of a planar surface.

3.1 Mandatory Part

You can start by modifying your RANSAC implementation from Lab 2. Rather than estimating an affine transformation, you should implement a function that estimates the homography between four correspondences (see the DLT algorithm in the reference literature by Hartley and Zisserman). When implementing the DLT algorithm, you should apply *normalization* to the input points. For each set of points you should:

- Translate the points so that their centroid is at the origin.
- Scale the points so that the average distance from the origin is $\sqrt{2}$.

This normalization is done using a similarity transform matrix T , which is applied to all input points before running DLT. After estimating the homography H' from the normalized points, you obtain the final homography as:

$$H = T_{\text{target}}^{-1} \cdot H' \cdot T_{\text{source}}.$$

Given your RANSAC-based method for estimating the homography between two images, implement the following pipeline:

- For a given set of images, extract SIFT features in each image. Next, match SIFT features between pairs of images and estimate a homography for each pair. Store each estimated homography, together with the inlier matches to it. Use code from Lab 2 if possible (e.g., for feature extraction and matching).
- Select the image pair with the largest number of inliers as the starting pair. The first image thereby provides a global coordinate system and the homography H provides the mapping from the second image into the global coordinate system.⁶
- Iteratively add the other images: Among all images not yet included, select the image I that has the largest number of homography inliers with any of the images already included in the reconstruction. Let J be the corresponding image in the reconstruction and $H_{J,I}$ the homography from I to J . Use $H_{J,I}$ and the known mapping H_J from the coordinate system of J into the global coordinate system to compute the global homography H_I for image I . This step is the 2D equivalent to the "Extend motion" step from the Structure-from-Motion pipeline presented in the lecture.
- Finally, create a sparse map of the scene: For each image I in the reconstruction, use H_I to map all SIFT features found in I into the global coordinate system. Visualize the resulting 2D point cloud.

You can use the "graf" and "wall" (both showing changes in viewpoint), as well as the "bark" and "boat" (both showing zoom and rotation changes) datasets from [here](#) to evaluate your approach.

⁶Assuming that your homography maps pixels in the second into the first image. If this is not the case, you need to invert the homography.

What to include in the report. Your report should consist of the following:

- A short abstract describing the problem behind the project and outlining your solution.
- A “Method” section describing your approach: Provide pseudo code for your 2D Structure-from-Motion system and explain each step in your own words. This section should be about one page long.
- A “Experimental Evaluation” section describing the experiments you perform with your approach as well as the results of these experiments. Specifically,
 1. List the thresholds that are used in your RANSAC-based homography estimation for each dataset in a table. The thresholds include the inlier threshold that you use for each dataset and the corresponding maximum number of RANSAC iterations you run.
 2. Visualize the 2D sparse maps in the global coordinate for each dataset and indicate the number of the image regarded as global coordinate in the caption.⁷
 3. The dataset also provides a ground truth homography from the first image in the sequence to each other image. Use these ground truth homographies to create sparse maps and visualize them next to your maps. Make sure to visualize the sparse map in the same global coordinate as 2) for each dataset.
 4. For each dataset, measure the average residual between each sparse point in your map and the corresponding point position in the ground truth map. Report the results in a table.

3.2 Advanced Part

As a post-processing step, improve your 2D Structure-from-Motion pipeline by using the 2D equivalent of bundle adjustment.

- **Form feature tracks:** If feature i in image I and feature j in image J form an inlier correspondence, and if feature k in image K and feature j in image J form an inlier correspondence, then i , j , and k form a feature track. Extend this definition to include all images where consistent matches exist.
- **Estimate initial 2D point positions:** For each track, transform all observed feature positions into the global coordinate system using the corresponding homographies. Compute the global point position as the average of these transformed coordinates.

Once you have obtained a set of global 2D point estimates, perform a non-linear optimization to **jointly** refine the 2D coordinates of all global points, **and** the homographies (assume the bottom-right element is fixed to remove scale ambiguity).

The goal is to minimize the reprojection error for all observed features. You should solve this problem using the Gauss-Newton method, i.e. at each iteration n , compute a parameter update

$$\theta^{(n+1)} = \theta^n - (J^\top J)^{-1} J^\top \mathbf{r}$$

where θ includes all 2D point coordinates and homography parameters stacked into a single vector, and J is the Jacobian of the residual vector $\mathbf{r}$ with respect to θ .

⁷The sparse map simply means all estimated 2D points, shown in the coordinate system of the reference image.

Hint 1: You do not need to compute the Jacobian analytically. Instead, you can use **finite difference approximation**. For each parameter θ_k , approximate:

$$\frac{\partial \mathbf{r}}{\partial \theta_k} \approx \frac{\mathbf{r}(\theta_k + \epsilon) - \mathbf{r}(\theta_k)}{\epsilon}$$

You should only compute residuals for actual feature observations. Each residual depends only on one 2D point and one homography, so the Jacobian is sparse.

Hint 2: To reduce computational cost, use a random subset of 2D points and their observations when building the residuals and Jacobian.

After optimization, reconstruct the sparse 2D map by transforming all observed features from each image into the global coordinate system using the optimized homographies.

What to include in the report. Extend your report to include the following:

- Extend the “Method” section by a subsection by adding a description of the method that you implemented in your own words.
- Extend the “Experimental Evaluation” section by adding visualizations of the 2D maps generated with your more advanced pipeline. Also measure the errors in your 2D point positions with respect to the ground truth map you generated before.

3.3 Theoretical Part

Answer the following questions in your report. Use your own words and justify your answers.

(a) Analyze which types of image regions tend to generate strong and reliable local features in your pipeline, and which regions tend to generate weak or ambiguous features. Relate your observations explicitly to the properties of the detector and descriptor used in your implementation. For every discussed behavior, identify 1-2 detected feature points from your own data that demonstrate the effect.

(b) Analyze what types of point configurations or image structures are problematic when estimating homographies, and explain why. Focus in particular on how the geometric distribution, spatial coverage, and arrangement of the matched points affect the estimated homographies. Relate your discussion explicitly to your own homography estimation results. For 2-3 issues, provide explicit examples from your own estimated correspondences, inlier/outlier classifications, or homography results.

(c) Discuss whether the strongest pairwise matches also produce the most stable long-term feature tracks. Identify examples of feature points that match well locally, but fail to generate stable or useful feature tracks over longer sequences. (i) If you implemented the advanced part and computed feature tracks explicitly, support your discussion using concrete examples from your own feature tracks. (ii) If you did not implement the advanced part, instead identify feature points or image regions in your own data that you hypothesize would likely: (i) generate stable long-term tracks, (ii) generate unstable or fragmented tracks, and (iii) disappear quickly across frames. Note: Every discussed behavior or hypothesis must be supported using explicit examples from your own data.

4 Project IV: Diffusion Models

Diffusion probabilistic models have recently become popular in Computer Vision due to their ability to generate high-resolution realistic images. In this project, you will be diving deep into the actual working of such models. You will work your way through the math involved and the tricks used to train neural network models to accomplish the generative task using this method. Due to limited data and computation available, you will be working with simple toy data.

Notes

1. It is recommended to use [PyTorch](#) for mandatory part-2 and the advanced part.
2. [blog 1](#) and [blog 2](#) are excellent sources that can serve as tutorials for the details of how diffusion models work and how they can be implemented.

4.1 Introduction

As discussed in the lectures, a (denoising) diffusion model consists of 2 parts:

1. A pre-defined forward diffusion process q that gradually adds small amounts of Gaussian noise to the original data, until you end up with pure noise.
2. A learned reverse (denoising) model p_θ that removes the noise to eventually produce something that if the training is successful should look like a sample from the training distribution.

Assuming a data point to have been sampled from an original distribution $\mathbf{x}_0 \sim q(\mathbf{x})$. We sequentially add Gaussian noise to it at timesteps $1 \dots T$ as per the conditional distribution:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \quad (1)$$

where the step sizes are controlled by the variance schedule $\{\beta_t \in (0, 1)\}_{t=1}^T$.

4.1.1 Mandatory task 1

If somehow the reverse process $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ was available, one would be able to produce meaningful data samples by repeatedly applying this denoising process starting from pure noise. Here we will consider a simple 1D case: $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{x}_0; m, p)$ and the forward diffusion process as mentioned above in (1). Assuming that the distribution of $\mathbf{x}_0$ is known, the reverse conditional distribution $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ can be computed in closed form, using the Bayes rule and the product of Gaussian distributions.

- Choose your own parameters (m, p) for the distribution of $\mathbf{x}_0$ and sample from this. Choose m and p such that you do not get the standard normal distribution (since we want a distribution that is different to the distribution used for x_T).
- Run the forward diffusion process for a number of timesteps, $T = 50$. You can experiment with different variance schedules (eg. linear, quadratic, etc.), but for simplicity, you can start with a constant schedule such that $\beta_t = 0.25 \forall t \in \{1 \dots T\}$. By repeating this process for various inputs, verify that the values at time T approximately follow a standard normal distribution. Visualize a few forward trajectories and also the distribution of x_T computed over at least 100 trajectories starting from different realizations of $\mathbf{x}_0$ by plotting x_t against t for $t \in (1 \dots T)$.

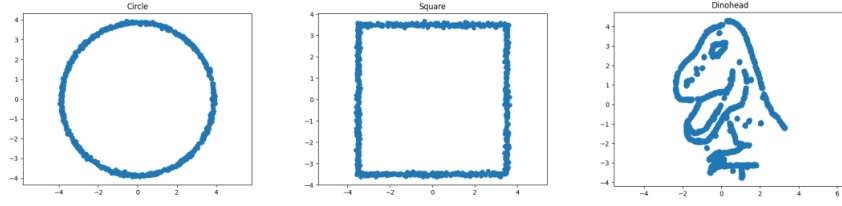


Figure 6: 2D point cloud datasets for mandatory task 2 - *Circle*, *Square* and *Dinohead*

- Compute the reverse process as a function of $\mathbf{x}_t$, $\mathbf{x}_0$ and t . You can take help from the derivations mentioned [here](#).
- Sample $\mathbf{x}_T$ from a standard normal distribution and repeatedly apply the reverse process computed above to obtain $\mathbf{x}_0$. Visualize a few of the reverse trajectories.

4.1.2 Mandatory task 2

In this task you will actually predict the $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ using a neural network with parameters θ . Hence this prediction/approximation will be termed as $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$. To learn this, you will closely follow the approach presented in [1]. You are provided sets of 2D points in the following shapes - a *Circle*, a *Square*, and *Dinohead* - an outline of a (rather simplistic) dinosaur head (Fig. 6). The task is to learn to produce something close to these starting from standard Gaussian noise. You are supposed to train a different model for each shape. This is not a great example to showcase the generative ability of diffusion methods, however, it is interesting in the sense that it allows understanding and implementing the intrinsic details of the method.

Use the following steps as guidelines to implement a training regime that produces the required output. While training the model for a particular shape example, do the following within each training loop:

- Sample a batch (randomly selected small subset) of points and for each point - a timestep, and noise - this is the true noise.
- Predict the 2D noise $\epsilon_\theta(\mathbf{x}_t, t)$ using $\mathbf{x}_t$ and t as inputs.
- Using squared error between the predicted and true noise as the loss, back-propagate and update the model parameters using gradient descent.

Other general tips:

- Instead of feeding the 2D points and the timesteps as inputs to the neural network directly, it is advisable to feed in *positional embeddings* of these inputs. High-frequency sinusoidal embeddings perform well. A python class extending `torch.nn.Module` for sinusoidal embeddings is provided in the *Stuff_for_Diffusion_ModelsProject* folder on Canvas. You may use it as reference as it might or might not match your required format exactly.
- An MLP (multi-layer perceptron) of 3-4 layers should be able to accomplish this task, trained for enough steps. Training such an MLP with Adam optimizer and learning rate in the range of 10^{-3} to 10^{-4} has been found to be effective.

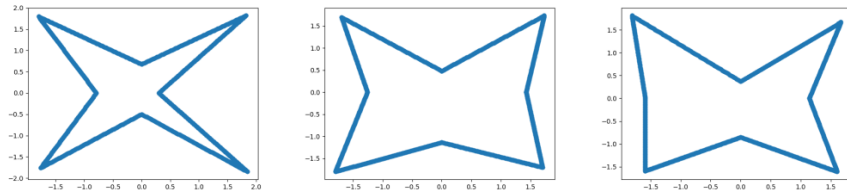


Figure 7: Examples of data samples from the *stars* dataset.

What to include in the report

- An introduction section describing the basics of diffusion probabilistic models, the underlying distributions, and the applications of such methods. This section should also explain the details of how the denoising task can be accomplished by a noise-predicting neural network. (About 1-2 pages)
- A method section with 2 subsections for the 2 mandatory tasks.
 1. In the first subsection, explain in your own words how the reverse process can be computed in closed form in the given setting.
 2. In the second subsection, explain in detail the steps involved in training your neural network and list the hyper-parameters used. Also, explain the steps involved in generating data at inference time.
- An experimental results section, again containing two subsections.
 1. For the first task, include the plots for forward, and backward trajectories and also the resulting distributions (of $\mathbf{x}_T$ in the forward process and $\mathbf{x}_0$ in the reverse process).
 2. For the second task, include scatter plots of the resulting 2D points from your inference process (reverse sampling from 2D Gaussian noise). Show results obtained at different stages of the training process (eg. after every 1k training steps). Also, include analysis over any other hyper-parameter whose impact you found interesting.

4.2 Advanced part

In the advanced part, you attempt to generate (somewhat) new data. Like 4.1.2, the data is still in the form of 2D points, however, instead of learning to generate points in a single pattern, you will try to generate different realizations of a shape template (examples shown in figure 7). You are provided 1024 data samples with 1024 points each. You can devise your own strategy for this or you can have roughly the same approach as before except the following major change.

1. Instead of your network producing noise for each point independently, the network would take in information about all points in the batch to estimate the noise for the batch.
Tip : Since the output noise should permute similarly upon the input points being permuted in a certain way, the additional information from other points should be something that is permutation invariant - like the average position of the points.

Upon successful implementation, you should be able to generate samples which resemble the input data (perhaps noisy versions of them).

4.3 Theoretical Part

Answer the following questions in your report. Use your own words and justify your answers.

(a) Compute the theoretical number of learnable parameters in your model. Show explicitly how the parameter count is derived for each relevant layer or component. Compare your theoretical result with the parameter count reported by the implementation, framework, or documentation, and discuss possible reasons for any differences. Discuss how the number of learnable parameters would change if you modified the architecture, for example by changing the hidden dimensionality or the number of hidden layers. Finally, discuss whether increasing the number of learnable parameters would necessarily improve the generated samples in this project.

(b) Estimate the computational complexity of your pipeline for (i) training, and (ii) inference/sampling. Clearly state all assumptions, including implementation details. Discuss which parts of the pipeline dominate the computational cost. Discuss why inference in diffusion models can be computationally expensive even when the underlying denoising model is relatively small.

(c) Identify 1-2 qualitative artifacts in your generated results in mandatory task 2 that deviates noticeably from the geometric structure or visual appearance of the corresponding training shape. Explain how the occurrence of these artifacts may be related to the assumption that points are generated independently. Discuss what types of geometric or structural relationships between points are difficult to capture under the independent-point assumption. Explain how such artifacts could potentially be reduced or avoided by training models on entire point clouds jointly, for example as in the advanced part of the project.

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *arXiv preprint arxiv:2006.11239*, 2020. [13](#)